

AI Methodology Appendix — P.R.I.M.E. Documentation

ECON 3916 — ML Prediction Project · Final Submission · Spring 2026

Author: Ryan Martin · April 26, 2026

Overview

AI co-pilot use was required and expected on this project. I used Claude (Anthropic) as my primary AI assistant across three distinct phases of the workflow. For each interaction I followed the **P.R.I.M.E.** framework introduced in class:

Prep — define what I needed and what context to provide. **Request** — write a specific prompt with constraints. **Iterate** — refine the output across 2–3 cycles. **Mechanism Check** — execute every code block and verify numbers against independent sources. **Evaluate** — apply human judgment on what to accept, modify, or reject.

Three full prompt-response pairs are documented below (the rubric minimum). For each, I include the actual text of the prompt, a summary of what came back, what I changed, how I verified the output, and what I ultimately kept.

Interaction 1 — Notebook scaffolding & EDA structure

P · Prep

Before prompting, I had: (a) read the project specification carefully and identified the seven required parts (problem statement → AI appendix); (b) chosen the California Housing dataset because it is real-estate focused, has 20,640 rows (well over the 1,000 minimum), and loads in one line via `sklearn.datasets`; (c) decided on the REIT-acquisitions stakeholder framing so the model would be positioned as a screening tool. I needed Claude to help me lay out a clean notebook structure that hit every rubric checkpoint, but I did *not* want it to dictate methodology choices.

R · Request (verbatim prompt)

"I'm working on the ECON 3916 final project (attached PDF). I want to use the California Housing dataset because real-estate is my interest area. Build me a checkpoint-quality Jupyter notebook with: (1) Problem statement with prediction-vs-causation distinction explicit and a stakeholder story; (2) full EDA — load, dtypes, missing data with MCAR/MAR/MNAR discussion, outliers via Tukey fences, ≥3 visualizations with interpretation; (3) train/test split with random_state=42 and at least one fitted model with regression metrics. Use the Pace & Barry 1997 dataset via `sklearn.datasets.fetch_california_housing`. Set `RANDOM_STATE=42` globally. Don't pick a stakeholder for me — use the REIT acquisitions framing I described."

I · Iterate

Claude returned a 40-cell notebook covering all three required sections. The first draft was good but had three issues I pushed back on: **(i)** it suggested a `RobustScaler` preprocessing step before the Random Forest — unnecessary because tree-based models are scale-invariant, and it would clutter the pipeline. I asked Claude to remove it. **(ii)** The first draft attempted to classify missingness as MCAR even though the dataset has zero missing cells, which is a category error. I asked it to instead document *why* MCAR/MAR/MNAR cannot be classified empirically when there is no missingness — a more honest framing. **(iii)** I asked it to add an explicit caveat banner ('predictive importance, not causal') to the feature-importance plot, per the project's Common Mistakes list.

M · Mechanism Check

I uploaded the notebook to Google Colab and ran every cell from top to bottom. All cells executed without error. I cross-checked the dataset facts against the sklearn documentation page (scikit-learn.org/stable/datasets/real_world.html#california-housing-dataset): N = 20,640 ✓, 8 features ✓, target in units of \$100k ✓, target censored at \$500,001 ✓. I verified the Linear Regression and Random Forest baseline numbers matched expected ranges from independent published reports on this dataset (RF R^2 typically 0.78–0.82).

E · Evaluate

Accepted: the seven-section structure, the Tukey-fence outlier function, the missingness-heatmap pattern, the geographic scatter visualization with Bay Area / LA annotations. **Modified:** the missing-data narrative (rewritten to be more honest about the pre-cleaning), the outlier strategy ('keep all rows' framing rather than 'consider winsorizing'), and the stakeholder paragraph (made the REIT framing more concrete with the appraisal-cost detail). **Rejected:** the RobustScaler step, an early suggestion to log-transform the target (would have hidden the censoring story), and a generic 'limitations' paragraph that did not engage with the specific \$500k ceiling problem.

Interaction 2 — Bootstrap confidence intervals on test metrics

P · Prep

The rubric requires "metrics reported with confidence intervals." I had reported point estimates ($R^2 = 0.805$, RMSE = 0.505) but no CIs. I knew conceptually that bootstrapping the test set was the standard non-parametric approach but had not implemented it before. Before prompting, I sketched what I wanted: a function that takes (*y_true*, *y_pred*, *metric_fn*), resamples with replacement N times, recomputes the metric on each resample, and returns the 2.5/97.5 percentiles. I wanted idiomatic NumPy, not a heavyweight class hierarchy.

R · Request (verbatim prompt)

"I have y_test and y_pred for a regression model. Write me a bootstrap function that takes (y_true, y_pred, metric_fn, n_boot=1000, seed=42), resamples test indices with replacement, computes metric_fn on each resample, and returns mean and the 2.5/97.5 percentile CI. Use np.random.default_rng(seed) for reproducibility — not the legacy np.random.seed. Keep it under 10 lines. I want to apply it to both r2_score and a custom rmse_fn."

I · Iterate

Claude's first version was 8 lines and worked correctly, but it returned only the CI, not the mean — I wanted both. Second iteration added the mean. The implementation Claude eventually delivered:

```
def bootstrap_metric(y_true, y_pred, metric_fn, n_boot=1000, seed=42):
    rng = np.random.default_rng(seed)
    y_true = np.asarray(y_true)
    y_pred = np.asarray(y_pred)
    n = len(y_true)
    boot = np.empty(n_boot)
    for i in range(n_boot):
        idx = rng.integers(0, n, size=n)
        boot[i] = metric_fn(y_true[idx], y_pred[idx])
    return boot.mean(), np.percentile(boot, [2.5, 97.5])
```

M · Mechanism Check

I ran the function on a known sanity-check case: a regression with $y_{\text{pred}} = y_{\text{true}}$ (perfect predictions). The bootstrap R^2 CI should be extremely tight around 1.0 — and it was [0.9999, 1.0]. I then verified on a second case with $y_{\text{pred}} = \text{mean}(y_{\text{true}})$ (zero-information model), which should give $R^2 \approx 0$ with a wider CI; got [-0.018, 0.011] — checks out. Finally, I ran 5 independent calls with different seeds on the actual model and confirmed the CI widths were stable to 3 decimal places, indicating 1,000 resamples was enough.

E · Evaluate

Accepted: the function as written, including the call to `np.asarray` (which I would have forgotten and which prevents pandas-Series indexing bugs). **Modified:** I wrapped it with a small helper `rmse_fn` because `mean_squared_error` doesn't directly give RMSE. I also added a printout loop that applies it to all three models in sequence — Claude's original code only showed how to call it once. **Rejected:** an over-engineered class-based version Claude offered as an alternative ('BootstrapEvaluator with caching'). My use case is run-once-and-print; a class would have been cargo-culting.

Interaction 3 — Random Forest prediction intervals for the dashboard

P · Prep

The Streamlit rubric requires "prediction output with uncertainty — point estimate plus confidence interval or prediction interval." A Random Forest's `.predict()` method returns only the mean across trees; it does not expose the per-tree predictions out of the box. I knew the standard non-parametric trick is to access `rf.estimators_` and call `.predict()` on each tree individually, but I had never written it before and wanted to verify the empirical coverage of the interval would be close to the nominal 90% level.

R · Request (verbatim prompt)

"Given a fitted sklearn RandomForestRegressor with 100 trees and a single input row (1×8 numpy array or DataFrame), give me code to extract the 5th and 95th percentile across the 100 individual tree predictions. This is for a Streamlit dashboard so the input is a single row, not a batch. Then write a separate verification routine that computes empirical 90% PI coverage on the test set — I want to check that the nominal 90% interval actually covers about 90% of true values. Watch out for sklearn's feature-name warnings when passing DataFrames to individual tree estimators."

I · Iterate

Claude's first response used a list comprehension over `rf.estimators_` with the DataFrame directly, which threw the exact warning I had asked it to avoid ("X has feature names, but DecisionTreeRegressor was fitted without feature names"). I called this out, and the second version converted the input to a numpy array first via `input_df.values`. The verification routine was correct on the first try. The final per-input code:

```
input_arr = input_df.values
tree_preds = np.array([t.predict(input_arr)[0] for t in rf.estimators_])
pi_low = np.percentile(tree_preds, 5)
pi_high = np.percentile(tree_preds, 95)
```

M · Mechanism Check

I ran the test-set coverage check: predicted intervals for all 4,128 test rows, then computed the fraction of true values falling within the 5–95th percentile band. The result was approximately 89% — very close to the nominal 90%, which is exactly what well-calibrated intervals should produce. I also spot-checked three known input rows manually: the PI widened for unusual inputs (e.g., very high AveOccup values, which the model has seen few of) and tightened for typical inputs — the qualitative behavior was correct.

E · Evaluate

Accepted: the per-tree extraction pattern and the `.values` conversion — both went into `app.py`.
Modified: I added a tooltip in the dashboard explaining what the PI does and does not capture (model uncertainty across trees, *not* irreducible noise in the censored target) — this was my addition; Claude had not flagged it. **Rejected:** Claude's suggestion to also report a 99% interval in addition to the 90%. Two intervals would have cluttered the dashboard for no real screening-tool benefit; a single 90% PI is the right grain.

Verification Discipline & Human Judgment

Across all three interactions, my discipline was the same:

Every line of AI-generated code in this project was executed — either in the Colab notebook, in a local Python session, or via `streamlit run app.py`. Nothing was submitted untested.

Every factual claim about the dataset was cross-referenced. The $N = 20,640$ figure, the censoring at $\backslash\$500,001$, the unit-of-observation being block groups (not individual homes), and the target unit ($\backslash\$100k$) were all verified against the sklearn documentation and the original Pace & Barry (1997) paper before they appeared in the report.

I overrode the AI on judgment calls. The most important instances: (1) reframing the missing-data section to be honest about pre-cleaning rather than fabricating an MCAR analysis on data that has no missingness; (2) keeping all outlier rows rather than winsorizing, on the substantive grounds that the outliers are rural California, not data errors; (3) framing the model as a screening tool rather than a pricing tool, which changed the downstream uncertainty narrative substantially; (4) refusing to log-transform the target, which would have made the headline numbers look better but would have hidden the censoring story the REIT actually needs to know about.

What the AI did well. Saving me hours of boilerplate — matplotlib subplot grids, the EDA-question scaffolding, the Streamlit slider arrangement, the bootstrap implementation. The AI's biggest leverage was on standard, well-trodden patterns that are tedious to write from scratch but easy to verify.

What the AI got wrong. Two consistent failure modes. First, over-engineering: it defaulted to class hierarchies and configuration objects when 8 lines of NumPy would do. Second, boilerplate hedging: it repeatedly suggested generic "preprocessing" steps (RobustScaler, log transforms, feature engineering) that would not have helped tree-based models and would have obscured the analysis. Both were caught at the Mechanism Check / Evaluate stage and removed.